

Лабораторная работа №7

Реализация формы регистрации и входа для пользователя и администратора

Содержание

1. Цель работы	3
2. Основные компоненты реализации	4
2.1. Интерфейс регистрации и входа	4
3. DTO (Data Transfer Object)	10
3.1. Зачем нужны DTO?	10
4. Контроллер регистрации	12
4.1. JWT (JSON Web Token)	12
5. Реализация ViewModel для формы регистрации	19
6. Контроллер OrderController	22
7. Реализация ProductController	24
8. ApiService	27
9. Program.cs	31
10. Реализация интерфейса	34
11. Вопросы по лабораторной работе №7	36

List of Listings

2.1	Разметка формы входа (LoginWindow.xaml)	5
2.2	LoginWindow.xaml.cs	6
2.3	Разметка окна регистрации (RegisterWindow.xaml)	7
2.4	Методы формы регистрации (RegisterWindow.xaml.cs)	8
2.5	Разметка окна успешной регистрации (SuccessWindow.xaml)	9
2.6	SuccessWindow.xaml.cs	9
4.1	Контроллер регистрации (AuthController.cs)	14
5.1	Класс RegisterViewModel	21
7.1	Контроллер ProductController.cs	26

1. Цель работы

Изучить принципы построения пользовательского интерфейса с возможностью регистрации, входа и разграничением доступа между ролями пользователя и администратора в приложении на WPF. Сформировать применение JWT-аутентификации, организовать ролевой доступ (только Admin может добавлять/удалять/изменять), а также реализовать регистрацию и авторизацию пользователей.

В современных приложениях важным элементом является наличие системы аутентификации (входа) и регистрации, которая позволяет пользователям взаимодействовать с приложением, а администраторам — управлять данными и пользователями.

Важно: необходимо запускать код данной работы только после успешного запуска проекта ASP.net, внося в него все необходимые изменения.

Ключевые элементы:

- Регистрация пользователя — ввод персональных данных и сохранение в системе.
- Аутентификация — проверка логина и пароля при входе.
- Роль (role-based access control) — модель разграничения доступа, при которой действия пользователя зависят от его роли.

Для реализации этих функций используется архитектура MVVM (Model-View-ViewModel), что обеспечивает разделение бизнес-логики и интерфейса, облегчает поддержку и масштабирование проекта. Для работы будут использоваться приложение WPF, разработанное в лабораторной №6, и ASP.NET API, разработанное в лабораторной №5.

2. Основные компоненты реализации

2.1. Интерфейс регистрации и входа

Для реализации регистрации и входа в приложение необходимо создать два окна: окно входа (логина) и окно регистрации. Они связаны с соответствующими ViewModel, в которых реализуется логика обработки ввода. Для добавления нового окна необходимо нажать правой кнопкой на библиотеке классов и выбрать. Добавить → Окно (WPF). Назвать файл LoginWindow.xaml.

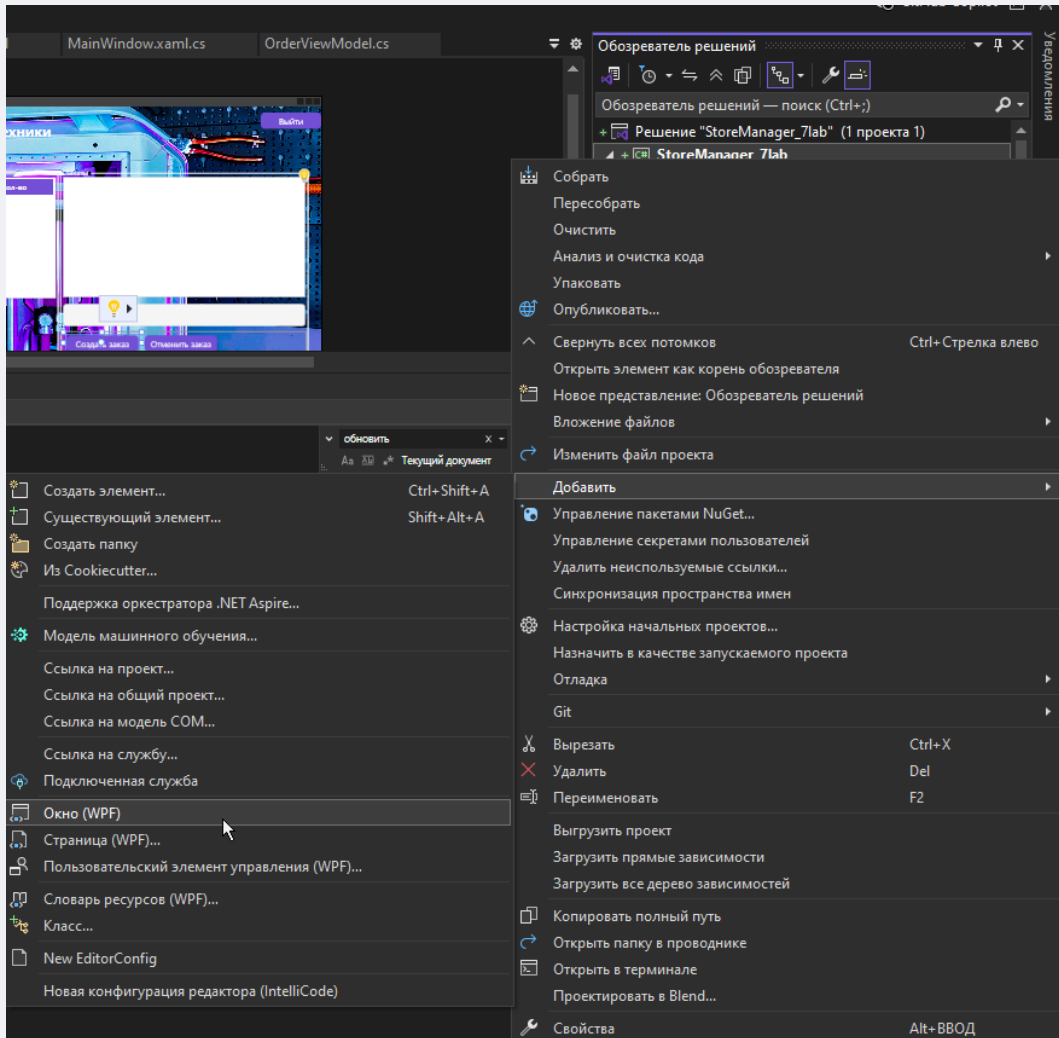


Рисунок 2.1 – Добавление нового окна в приложение

В листинге 2.1 приведен пример разметки для окна входа пользователя (логина).

```

1 <StackPanel Margin="30" VerticalAlignment="Center">
2     <TextBlock Text="Вход"
3         FontSize="24"
4         FontWeight="Bold"

```

```

5         HorizontalAlignment="Center"
6         Margin="0,0,0,20" />
7
8     <TextBlock Text="Логин:" FontWeight="SemiBold"/>
9     <TextBox Text="{Binding Username}" Margin="0,5,0,5" Padding="5"/>
10
11    <TextBlock Text="Пароль:" FontWeight="SemiBold"/>
12    <PasswordBox x:Name="PasswordBox"
13
14        Margin="0,5,0,10"
15        Padding="5"
16        PasswordChanged="PasswordBox_PasswordChanged" />
17
18    <Button Content="Войти"
19        Style="{StaticResource PurpleButton}"
20        Command="{Binding LoginCommand}" />
21
22
23    <!-- Ссылка на регистрацию -->
24    <TextBlock HorizontalAlignment="Left" FontSize="12">
25        <Run Text="Нет аккаунта?" />
26        <Run Text="Регистрация"
27
28            Foreground="#8703ab"
29            TextDecorations="Underline"
30            Cursor="Hand"
31            MouseDown="RegisterText_MouseDown" />
32    </TextBlock>
33 </StackPanel>

```

Листинг 2.1 – Разметка формы входа ([LoginWindow.xaml](#))

Затем необходимо добавить следующий листинг в `LoginWindow.xaml.cs` (листинг 2.2).

```

1 namespace StoreManager_7lab
2 {
3
4     public partial class LoginWindow : Window
5     {
6         private void RegisterText_MouseDown(object sender, MouseButtonEventArgs e)
7         {
8             var reg = new RegisterWindow();
9             reg.ShowDialog();
10
11             this.Close();
12         }
13         public LoginWindow()
14         {
15             InitializeComponent();

```

```

16         if (DataContext is LoginViewModel vm)
17         {
18             vm.LoginSucceeded += () => this.Close();
19         }
20     }
21
22     private void PasswordBox_PasswordChanged(object sender, RoutedEventArgs e)
23     {
24         if (DataContext is LoginViewModel vm)
25             vm.Password = PasswordBox.Password;
26     }
27 }
28 }

```

Листинг 2.2 – LoginWindow.xaml.cs

В данном листинге (см. листинг 2.1) учитывается, что после входа пользователя в приложение данная форма будет закрыта, а также есть возможность открыть окно регистрации по ссылке, нажав на соответствующий текст в форме. Далее будет рассмотрено создание формы регистрации пользователей. В данном случае для этого был создан файл RegisterWindow.xaml. Пример разметки для формы регистрации приведен в листинге 2.3.

```

1 <Window x:Class="StoreManager_7lab.RegisterWindow"
2     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4     Title="Регистрация" <!-- Added Window tag for completeness -->
5     <Grid Margin="30">
6         <Grid.RowDefinitions>
7             <RowDefinition Height="Auto"/>
8             <RowDefinition Height="Auto"/>
9             <RowDefinition Height="Auto"/>
10            <RowDefinition Height="Auto"/>
11            <RowDefinition Height="Auto"/>
12            <RowDefinition Height="*" />
13        </Grid.RowDefinitions>
14
15        <TextBlock Grid.Row="0"
16            Text="Регистрация"
17            FontSize="24"
18            FontWeight="Bold"
19            HorizontalAlignment="Center"
20            Margin="0,0,0,20"
21            Foreground="#8703ab" />
22
23        <TextBlock Grid.Row="1" Text="Имя пользователя:" />
24        <TextBox Grid.Row="2"
25            Margin="0,5,0,10"
26            Padding="3"

```

```

27         Text="{Binding Username, UpdateSourceTrigger=PropertyChanged}"
28         Background="#2A2A2A"
29         BorderBrush="#8703ab"
30         Foreground="White" />
31
32     <TextBlock Grid.Row="3" Text="Пароль:" />
33     <PasswordBox Grid.Row="4"
34         Margin="0,5,0,10"
35         x:Name="PasswordBox"
36         Padding="3"
37         PasswordChanged="PasswordBox_PasswordChanged"
38         Background="#2A2A2A"
39         BorderBrush="#8703ab"
40         Foreground="White" />
41
42     <StackPanel Grid.Row="5" Orientation="Vertical">
43         <ComboBox ItemsSource="{Binding Roles}"
44             SelectedItem="{Binding SelectedRole}"
45             Foreground="#8703ab"
46             Margin="0,0,0,10" />
47
48         <!-- Ссылка на вход -->
49         <TextBlock HorizontalAlignment="Left" FontSize="12">
50             <Run Text="Уже зарегистрированы?" />
51             <Run Text="Войти"
52                 Foreground="#8703ab"
53                 TextDecorations="Underline"
54                 Cursor="Hand"
55                 MouseDown="RegisterText_MouseDown" /> <!-- Assumed this is for
56                 ↪ LoginText_MouseDown -->
57         </TextBlock>
58
59         <Button Content="Зарегистрироваться"
60             Command="{Binding RegisterCommand}"
61             Background="#8703ab"
62             Foreground="White"
63             FontWeight="Bold"
64             Height="35" />
65     </StackPanel>
66 </Grid>
</Window>

```

Листинг 2.3 – Разметка окна регистрации ([RegisterWindow.xaml](#))

Далее необходимо реализовать методы, отвечающие за функционирование кнопок самой формы (листинг 2.4).

```

1 namespace StoreManager_7lab
2 {

```

```
3 public partial class RegisterWindow : Window
4 {
5     public RegisterWindow()
6     {
7         InitializeComponent();
8
9         if (DataContext is RegisterViewModel vm)
10         {
11             vm.RegistrationSucceeded += OnRegistrationSucceeded;
12         }
13     }
14
15     private void PasswordBox_PasswordChanged(object sender, RoutedEventArgs e)
16     {
17         if (DataContext is RegisterViewModel vm)
18         {
19             vm.Password = ((PasswordBox)sender).Password;
20         }
21     }
22     // Changed from RegisterText_MouseDown to LoginText_MouseDown based on context
23     private void LoginText_MouseDown(object sender, MouseButtonEventArgs e)
24     {
25         var loginWindow = new LoginWindow();
26         loginWindow.Show();
27
28         this.Close(); // закрываем текущее окно регистрации
29     }
30     private void OnRegistrationSucceeded()
31     {
32         Application.Current.Dispatcher.Invoke(() =>
33         {
34             // Show success message window before login window
35             var successWindow = new SuccessWindow();
36             successWindow.ShowDialog();
37
38             var loginWindow = new LoginWindow();
39             loginWindow.Show();
40             this.Close();
41         });
42     }
43 }
44 }
```

Листинг 2.4 – Методы формы регистрации ([RegisterWindow.xaml.cs](#))

Также можно добавить пользовательский вариант окна уведомления об успешной регистрации пользователя. Необходимо добавить новое окно и назвать его `SuccessWindow.xaml`. В листинге 2.5 приведена разметка данного окна.


```
1 <Window x:Class="StoreManager_7lab.SuccessWindow"
2       xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3       xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4       Title="Успех"
5       Height="150" Width="300"
6       WindowStartupLocation="CenterScreen"
7       Background="#8703ab">
8     <Grid>
9       <TextBlock Text="Регистрация успешна!"
10                Foreground="White"
11                FontSize="18"
12                FontWeight="Bold"
13                HorizontalAlignment="Center"
14                VerticalAlignment="Center"/>
15       <!-- Removed OkButton_Click as it is not in the XAML -->
16     </Grid>
17 </Window>
```

Листинг 2.5 – Разметка окна успешной регистрации ([SuccessWindow.xaml](#))

```
1 namespace StoreManager_7lab
2 {
3     public partial class SuccessWindow : Window
4     {
5         public SuccessWindow()
6         {
7             InitializeComponent();
8             // Example: Close after a delay or add a button programmatically
9             // For simplicity, this window might close itself or be closed by parent logic
10        }
11
12        // This method was in the original text but no button was in XAML.
13        // If a button is added, its Click event can be wired here.
14        // private void OkButton_Click(object sender, RoutedEventArgs e)
15        // {
16        //     this.Close();
17        // }
18    }
19 }
```

Листинг 2.6 – [SuccessWindow.xaml.cs](#)

3. DTO (Data Transfer Object)

DTO (Data Transfer Object) — это объект, предназначенный для передачи данных между слоями приложения или между клиентом и сервером. В контексте Web API DTO особенно полезны для отделения внутренней модели базы данных от той, что используется внешними клиентами (например, WPF или другим API-клиентом).

3.1. Зачем нужны DTO?

- **Изоляция модели базы данных:** DTO позволяет не раскрывать клиенту внутреннюю структуру сущностей, связанных с БД. Это снижает риск случайного или преднамеренного изменения чувствительных данных.
- **Упрощение данных:** Иногда внутренние модели содержат навигационные свойства, связанные сущности и прочее — всё это не всегда нужно на клиенте. DTO может содержать только необходимую информацию.
- **Валидация и контроль:** DTO позволяет настроить отдельные правила валидации, которые могут отличаться от ограничений модели БД.
- **Безопасность:** DTO защищает от уязвимостей, когда злоумышленник может отправить больше данных, чем нужно (например, роль пользователя в User, которую нельзя задавать напрямую через API без проверки).

Далее будут рассмотрены DTO, применяемые в данной лабораторной для входа и регистрации пользователей.

Пример 1: LoginDto:

```

1 namespace StoreManagerApi.Models
2 {
3     public class LoginDto
4     {
5         public string Username { get; set; } // Логин пользователя
6         public string Password { get; set; } // Пароль
7     }
8 }
```

LoginDto используется только для входа в систему. Не содержит Id, Role или других свойств, которые есть в полной модели User. Это делает передачу данных более безопасной и упрощённой.

Пример 2: UserRegisterDto (если бы он был)

```
1 public class UserRegisterDto
2 {
3     public string Username { get; set; }
4     public string Password { get; set; }
5     public string Role { get; set; } // Добавлено для регистрации, если роль передается
6 }
```

UserRegisterDto используется для регистрации. Он позволяет задать имя, пароль и роль, не включая другие свойства модели User, такие как Id, который создаётся автоматически.

Почему не стоит использовать саму модель User:

- Она может содержать больше данных, чем нужно;
- Она может подвергаться сериализации/десериализации, что может открыть доступ к Role или Id;
- Это нарушает принцип единой ответственности: DTO отвечает только за передачу конкретных данных.

4. Контроллер регистрации

Для реализации данного контроллера необходимо в приложении ASP.NET в папке `Controllers` создать файл `AuthController.cs`. Данный контроллер отвечает за:

1. Регистрацию пользователей (POST `/api/auth/register`);
2. Получение всех пользователей (служебный метод — GET `/api/auth/all`);
3. Аутентификацию (логин) с выдачей JWT-токена (POST `/api/auth/login`).

Он работает совместно с Entity Framework Core (через `StoreDbContext`) и системой JWT-аутентификации.

4.1. JWT (JSON Web Token)

JWT — это стандарт для безопасной передачи данных между двумя сторонами как JSON-объект. Он используется для авторизации и передачи удостоверений пользователя. В ASP.NET Core JWT применяется для защиты API и проверки прав пользователя. JWT состоит из трёх частей:

- Header — указывает тип токена и алгоритм подписи (обычно HMAC SHA256).
- Payload — содержит данные (претензии), например имя пользователя, его роль.
- Signature — создаётся на основе header, payload и секретного ключа.

Преимущества JWT:

1. Без состояния (stateless).
2. Передаётся через заголовки HTTP (Authorization: Bearer).
3. Подписан и проверяется на подлинность.

Как устроен JWT в этом коде?

1. JWT формируется вручную в методе `Login`:
2. Находится пользователь по имени и паролю;
3. Формируется список претензий (claims) — имя и роль;
4. Создаётся симметричный ключ с помощью `SymmetricSecurityKey`;
5. Подписывается токен с использованием HMAC SHA256;

6. Возвращается токен клиенту в формате JSON.

Полный листинг данного контроллера рассмотрен в листинге 4.1.

```
1 using Microsoft.AspNetCore.Mvc;
2 using Microsoft.EntityFrameworkCore; // Предполагается, что StoreDbContext здесь
3 using Microsoft.AspNetCore.Identity; // Для PasswordHasher
4 using System.Linq;
5 using System.Security.Claims;
6 using Microsoft.IdentityModel.Tokens;
7 using System.Text;
8 using System.IdentityModel.Tokens.Jwt;
9 using System; // Для DateTime
10 // Assuming StoreDbContext and User are defined elsewhere
11 // namespace StoreManagerApi.Data { public class StoreDbContext : DbContext { /* ... */ } }
12 // namespace StoreManagerApi.Models { public class User { /* ... */ } }
13 // namespace StoreManagerApi.Models { public class LoginDto { /* ... */ } }
14
15
16 [ApiController]
17 [Route("api/[controller]")]
18 public class AuthController : ControllerBase
19 {
20     private readonly StoreDbContext _context; // StoreDbContext должен быть определен
21     // private readonly UserManager<User> _userManager; // Если используется ASP.NET Core Identity
22     // private readonly SignInManager<User> _signInManager; // Если используется ASP.NET Core
23     // Identity
24
25     public AuthController(StoreDbContext context) // UserManager<User> userManager
26     {
27         _context = context;
28         // _userManager = userManager;
29     }
30
31     // POST: api/auth/register
32     // Регистрирует нового пользователя с хешированием пароля
33     [HttpPost("register")]
34     public IActionResult Register(User user) // User должен быть моделью с Username, Password, Role
35     {
36         // Проверка: если пользователь уже существует, вернуть ошибку
37         if (_context.Users.Any(u => u.Username == user.Username))
38             return BadRequest("Такой пользователь уже существует");
39
40         var hasher = new PasswordHasher<User>();
41         user.Password = hasher.HashPassword(user, user.Password); // Хешируем пароль
42
43         _context.Users.Add(user); // Добавляем пользователя в БД
44         _context.SaveChanges(); // Сохраняем изменения
45
46         return Ok("Регистрация прошла успешно");
47     }
48
49     // GET: api/auth/all
```

```
49 // Возвращает всех пользователей (можно использовать для теста)
50 [HttpGet("all")]
51 public IActionResult GetAllUsers()
52 {
53     return Ok(_context.Users.ToList()); // Возвращаем список всех пользователей
54 }
55
56 // POST: api/auth/login
57 // Авторизация: проверка логина и пароля, выдача JWT-токена
58 [HttpPost("login")]
59 public IActionResult Login(LoginDto dto) // LoginDto должен быть определен
60 {
61     // Поиск пользователя по логину
62     var user = _context.Users.FirstOrDefault(u => u.Username == dto.Username);
63     if (user == null)
64         return Unauthorized("Неверный логин");
65
66     var hasher = new PasswordHasher<User>();
67     // Сравнение введенного пароля с хешем из БД
68     var result = hasher.VerifyHashedPassword(user, user.Password, dto.Password);
69
70     if (result == PasswordVerificationResult.Failed)
71         return Unauthorized("Неверный пароль");
72
73     // Создание набора claims (утверждений) о пользователе
74     var claims = new[]
75     {
76         new Claim(ClaimTypes.Name, user.Username), // Имя пользователя
77         new Claim(ClaimTypes.Role, user.Role)      // Его роль (Admin/Customer)
78     };
79
80     // Генерация ключа и токена
81     var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes("THIS_IS_MY_SUPER_SECRET_KEY_123"
82     ↪ "4567890")); // Ключ должен быть безопасным и из конфигурации
83     var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
84
85     var token = new JwtSecurityToken(
86         claims: claims,
87         expires: DateTime.Now.AddHours(2), // Время действия токена
88         signingCredentials: creds);        // Подпись токена
89
90     // Отправка токена клиенту
91     return Ok(new
92     {
93         token = new JwtSecurityTokenHandler().WriteToken(token), // строка-токен
94         role = user.Role // роль для клиента
95     });
96 }
```

Листинг 4.1 – Контроллер регистрации ([AuthController.cs](#))

Далее будет рассмотрен поэтапно полный листинг контроллера регистрации.

```
1 [ApiController]
2 [Route("api/[controller]")]
3 public class AuthController : ControllerBase
```

- Атрибут `ApiController` говорит ASP.NET, что это контроллер Web API;
- `Route("api/[controller]")` означает маршрут `api/auth`.

```
1 private readonly StoreDbContext _context;
2 public AuthController(StoreDbContext context)
3 {
4     _context = context;
5 }
```

- зависимость от базы данных внедряется через DI;
- используется контекст для чтения и записи пользователей.

Внедрение зависимости от базы данных через Dependency Injection (DI) означает, что объект `StoreDbContext` не создаётся вручную в каждом контроллере, а предоставляется автоматически системой ASP.NET Core. Dependency Injection — это паттерн проектирования, при котором зависимости (например, классы, которые нужны контроллеру) передаются извне, а не создаются внутри класса. В контексте ASP.NET Core:

- `StoreDbContext` зарегистрирован как сервис в методе `builder.Services.AddDbContext<StoreDbContext>()` в `Program.cs`.
- когда создаётся экземпляр контроллера `AuthController`, ASP.NET Core автоматически передаёт туда экземпляр `StoreDbContext`.

Метод регистрации

```
1 [HttpPost("register")]
2 public IActionResult Register(User user)
3 {
4     // Проверка: если пользователь уже существует, вернуть ошибку
5     if (_context.Users.Any(u => u.Username == user.Username))
6         return BadRequest("Такой пользователь уже существует");
7
8     var hasher = new PasswordHasher<User>();
9     user.Password = hasher.HashPassword(user, user.Password); // Хешируем пароль
10 }
```

```
11     _context.Users.Add(user); // Добавляем пользователя в БД
12     _context.SaveChanges();   // Сохраняем изменения
13
14     return Ok("Регистрация прошла успешно");
15 }
```

- проверяет, существует ли уже пользователь;
- добавляет нового в базу данных;
- возвращает 200 ОК, если всё прошло успешно.

Этот код усиливает безопасность системы авторизации, добавляя хеширование паролей с использованием стандартного компонента ASP.NET Core — `PasswordHasher<T>`. Вместо того чтобы хранить пароли в базе данных в виде обычного текста (что крайне небезопасно), при регистрации пароль пользователя преобразуется в хеш, и только он сохраняется. При логине введенный пароль сравнивается с сохранённым хешем — если они совпадают, пользователь считается аутентифицированным.

Регистрация с хешированием:

```
1 var hasher = new PasswordHasher<User>();
2 user.Password = hasher.HashPassword(user, user.Password);
```

- `PasswordHasher<User>` — встроенный класс, который реализует надёжный алгоритм хеширования.
- `HashPassword()` — создаёт уникальный хеш для пароля. Даже одинаковые пароли разных пользователей будут иметь разные хеши.

После этого хеш сохраняется в базу вместо самого пароля.

Авторизация и проверка хеша:

```
1 var user = _context.Users.FirstOrDefault(u => u.Username == dto.Username);
2 if (user == null)
3     return Unauthorized ("Неверный логин");
4
5 var hasher = new PasswordHasher<User>();
6 var result = hasher.VerifyHashedPassword(user, user.Password, dto.Password);
7 if (result == PasswordVerificationResult.Failed)
8     return Unauthorized ("Неверный пароль");
```

- `VerifyHashedPassword()` проверяет, совпадает ли хеш из БД с паролем, введенным пользователем.

- Метод возвращает Success, Failed или SuccessRehashNeeded.

Получение всех пользователей

```
1 [HttpGet("all")]
2 public IActionResult GetAllUsers()
3 {
4     return Ok(_context.Users.ToList());
5 }
```

- возвращает список всех зарегистрированных пользователей;
- полезен только для отладки. Не должен быть открыт в боевом API.

Метод входа (Login)

```
1 [HttpPost("login")]
2 public IActionResult Login(LoginDto dto)
3 {
4     var user = _context.Users.FirstOrDefault(u => u.Username == dto.Username); // Пароль
4     ↪ проверяется ниже с хешем
5     if (user == null)
6         return Unauthorized("Неверный логин или пароль"); // Общее сообщение
7     // ... (проверка хеша пароля как выше) ...
```

- проверка имени пользователя и пароля (с хешем);
- если не найден: ошибка 401 Unauthorized.

```
1 var hasher = new PasswordHasher<User>();
2 // Сравнение введенного пароля с хешем из БД
3 var result = hasher.VerifyHashedPassword(user, user.Password, dto.Password);
4 if (result == PasswordVerificationResult.Failed)
5     return Unauthorized("Неверный пароль");
6
7 var claims = new[]
8 {
9     new Claim(ClaimTypes.Name, user.Username),
10    new Claim(ClaimTypes.Role, user.Role)
11 };
```

Формирование претензий (Claims) — имя и роль пользователя. Claims (утверждения) — это пары ключ-значение, которые представляют информацию о пользователе или субъекте токена. Они встраиваются в Payload JWT-токена и используются системой аутентификации и авторизации для принятия решений о доступе.

```
1 var key = new
  ↳ SymmetricSecurityKey(Encoding.UTF8.GetBytes("THIS_IS_MY_SUPER_SECRET_KEY_1234567890"));
2 var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
```

Создаётся ключ шифрования и подпись токена.

```
1 var token = new JwtSecurityToken(
2     claims: claims,
3     expires: DateTime.Now.AddHours(2),
4     signingCredentials: creds);
```

Собирается сам токен: данные + срок действия + подпись.

```
1     return Ok(new
2     {
3         token = new JwtSecurityTokenHandler().WriteToken(token),
4         role = user.Role
5     });
6 } // Закрытие метода Login
```

Возвращается JSON с токеном и ролью пользователя; Этот токен потом передаётся в заголовке Authorization: Bearer.

Пример успешного ответа

```
1 {
2     "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6I..",
3     "role": "Admin"
4 }
```

5. Реализация ViewModel для формы регистрации

Класс RegisterViewModel реализует интерфейс INotifyPropertyChanged и выступает как ViewModel для формы регистрации (например, RegisterWindow.xaml). Он содержит свойства, связанные с вводом (логин, пароль, роль), команды, вызываемые кнопками, и методы взаимодействия с API через ApiService. Реализация ViewModel для проекта WPF представлена в листинге 5.1.

```
1 using System;
2 using System.Collections.ObjectModel;
3 using System.ComponentModel;
4 using System.Runtime.CompilerServices;
5 using System.Threading.Tasks;
6 using System.Windows;
7 using System.Windows.Input;
8 // Предполагается наличие RelayCommand, ApiService, UserRegisterDto, SuccessWindow, LoginWindow
9
10 public class RegisterViewModel : INotifyPropertyChanged
11 {
12     // Поля, связанные с пользовательским вводом
13     private string _username;
14     public string Username
15     {
16         get => _username;
17         set { _username = value; OnPropertyChanged();
18             ↪ ((RelayCommand)RegisterCommand).RaiseCanExecuteChanged(); }
19     }
20     private string _password;
21     public string Password
22     {
23         get => _password;
24         set { _password = value; OnPropertyChanged();
25             ↪ ((RelayCommand)RegisterCommand).RaiseCanExecuteChanged(); }
26     }
27     public ObservableCollection<string> Roles { get; } = new() { "Customer", "Admin" };
28
29     private string _selectedRole;
30     public string SelectedRole
31     {
32         get => _selectedRole;
33         set { _selectedRole = value; OnPropertyChanged();
34             ↪ ((RelayCommand)RegisterCommand).RaiseCanExecuteChanged(); }
35     }
36     public ICommand RegisterCommand { get; } // Команда регистрации (привязана к кнопке)
37
38     public event Action RegistrationSucceeded; // Событие, вызываемое при успешной
39     ↪ регистрации
```

```

39
40     private readonly ApiService _api = new ApiService(); // Сервис взаимодействия с Web API
41
42     // Конструктор ViewModel
43     public RegisterViewModel()
44     {
45         // Привязка команды к методу регистрации и условию доступности
46         RegisterCommand = new RelayCommand(
47             async _ => await RegisterAsync(),
48             _ => CanRegister());
49     }
50     // Асинхронный метод регистрации пользователя
51     private async Task RegisterAsync()
52     {
53         try
54         {
55             // await _api.InitAsync(); // (необязательная инициализация клиента, если есть)
56
57             // Создание DTO с введёнными данными
58             var dto = new UserRegisterDto // UserRegisterDto должен быть определен
59             {
60                 Username = Username,
61                 Password = Password,
62                 Role = SelectedRole
63             };
64
65             await _api.RegisterAsync(dto); // Отправка на сервер
66
67             // Вызов события для закрытия окна и открытия окна входа из кода RegisterWindow.xaml.cs
68             RegistrationSucceeded?.Invoke();
69
70             // Закрытие текущего окна регистрации (это может быть сделано в обработчике
71             ↪ RegistrationSucceeded)
72             // foreach (Window window in Application.Current.Windows)
73             // {
74             //     if (window is RegisterWindow) // RegisterWindow должен быть определен
75             //     {
76             //         window.Close();
77             //         break;
78             //     }
79             // }
80             catch (Exception ex)
81             {
82                 // Сообщение об ошибке
83                 MessageBox.Show($"Ошибка регистрации: {ex.Message}");
84             }
85         }
86         // Условие для активации кнопки регистрации
87         private bool CanRegister() =>
88             !string.IsNullOrEmpty(Username) &&

```

```
89         !string.IsNullOrEmpty(Password) &&
90         !string.IsNullOrEmpty(SelectedRole);
91
92     // Реализация интерфейса INotifyPropertyChanged
93     public event PropertyChangedEventHandler PropertyChanged;
94     protected void OnPropertyChanged([CallerMemberName] string name = null) =>
95         PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
96 }
```

Листинг 5.1 – Класс `RegisterViewModel`

1. **RegisterAsync()** Метод `RegisterAsync()` реализует основной алгоритм регистрации пользователя:

- (a) Выполняется предварительная инициализация API-сервиса через вызов `InitAsync()` (если такой метод есть и он необходим).
- (b) Создаётся объект `UserRegisterDto`, содержащий данные, введённые пользователем: имя, пароль и выбранную роль.
- (c) С помощью метода `_api.RegisterAsync(dto)` данные отправляются на сервер в формате JSON, где происходит регистрация нового пользователя.
- (d) При успешной регистрации вызывается событие `RegistrationSucceeded`. В обработчике этого события (в `RegisterWindow.xaml.cs`) отображается окно подтверждения (`SuccessWindow`), после чего запускается окно входа (`LoginWindow`).
- (e) Активное окно регистрации (`RegisterWindow`) закрывается (также в обработчике события).

Метод обернут в конструкцию `try-catch`, обеспечивающую обработку исключений и отображение ошибок в форме диалогового окна.

2. **CanRegister()** `CanRegister()` определяет, можно ли активировать кнопку регистрации. Возвращает `true`, если все три поля (имя пользователя, пароль и роль) не пустые и не содержат только пробелы. В противном случае — `false`. Это предотвращает отправку неполных данных на сервер.
3. **Метод `OnPropertyChanged(string name)`** `OnPropertyChanged()` — вспомогательный метод, реализующий механизм уведомления об изменении свойств, требуемый интерфейсом `INotifyPropertyChanged`. Он используется для обновления интерфейса в режиме реального времени при изменении свойств `ViewModel`.

6. Контроллер OrderController

Контроллер `OrderController` — это важнейший компонент API, который управляет заказами в системе. Без него невозможно:

1. Получить список заказов.
2. Создать новый заказ.
3. Удалить заказ (при наличии соответствующих прав).

Он работает как интерфейс между клиентом (например, WPF-приложением) и базой данных, обрабатывая HTTP-запросы, выполняя валидацию данных и обеспечивая безопасность.

Ключевая часть:

```
1 [Authorize(Roles = "Admin")] // Требуется авторизация и роли Admin
2 [HttpDelete("{id}")]
3 public async Task<IActionResult> DeleteOrder(int id)
4 {
5     // ... логика удаления ...
6 }
```

Эти атрибуты обозначают, что данный метод удаления заказа:

- требует авторизации (`[Authorize]`);
- допускает доступ только пользователю с ролью `Admin` (`Roles = Admin`);
- будет вызван по маршруту `DELETE /api/order/id` (например, `/api/order/5`).

Таким образом, безопасность обеспечивается на уровне маршрута: обычный пользователь не сможет удалить заказ, даже если знает правильный адрес.

Основной код:

```
1 var order = await _context.Orders
2     .Include(o => o.Items) // Включаем связанные элементы заказа
3     .FirstOrDefaultAsync(o => o.Id == id);
```

В данном случае:

- `_context.Orders` — обращение к таблице заказов;
- `.Include(o => o.Items)` — указывает EF Core, что нужно включить связанные записи из таблицы `OrderItems` (позиции заказа). Без этого `Items` будут `null`;

- `.FirstOrDefaultAsync(o => o.Id == id)` — ищет первый заказ, чей Id совпадает с переданным. Если заказ не найден — вернёт `null`.

Затем выполняется проверка:

```
1 if (order == null)
2     return NotFound(); // Код 404, если заказ не найден
```

Если заказ не найден — возвращается код ответа 404 Not Found. Удаление заказа осуществляется с помощью метода `Remove`: `_context.Orders.Remove(order)`;

```
1 await _context.SaveChangesAsync(); // Сохранение изменений в БД
```

Сначала EF помечает объект как удалённый, а затем сохраняет изменения в базе. Благодаря каскадной настройке в `DbContext`, также удаляются связанные `OrderItem`.

7. Реализация ProductController

ProductController — контроллер, отвечающий за управление товарами в API. Он реализует CRUD-операции для ресурса Product, а также демонстрирует работу с авторизацией на основе ролей.

Основные возможности ProductController:

1. Позволяет всем пользователям просматривать список товаров и отдельные товары по ID.
2. Предоставляет администраторам (роль Admin) возможность добавлять, редактировать и удалять товары.
3. Использует базу данных через StoreDbContext, внедрённый через Dependency Injection.
4. Защищён с помощью атрибутов [Authorize] и [AllowAnonymous].

В листинге 7.1 приведен полный код данного контроллера с комментариями.

```
1 using Microsoft.AspNetCore.Mvc;
2 using Microsoft.EntityFrameworkCore;
3 using System.Collections.Generic;
4 using System.Threading.Tasks;
5 using Microsoft.AspNetCore.Authorization; // Для атрибутов авторизации
6 // Предполагается, что Product и StoreDbContext определены
7 // namespace StoreManagerApi.Models { public class Product { /* ... */ } }
8
9 namespace StoreManagerApi.Controllers
10 {
11     [ApiController]
12     [Route("api/[controller]")] // Базовый маршрут: api/Product
13     public class ProductController : ControllerBase
14     {
15         private readonly StoreDbContext _context;
16
17         // Внедрение зависимости через конструктор
18         public ProductController(StoreDbContext context) => _context = context;
19
20         // =====
21         // GET: /api/Product
22         // Возвращает список всех товаров
23         // Доступен без авторизации
24         // =====
25         [AllowAnonymous] // Разрешает доступ без токена
26         [HttpGet]
27         public async Task<ActionResult<IEnumerable<Product>>> GetAll() =>
28             await _context.Products.ToListAsync();
29
30         // =====
31         // GET: /api/Product/5
32         // Получает один товар по его ID
```



```

33     // Доступен без авторизации
34     // =====
35     [AllowAnonymous]
36     [HttpGet("{id}")]
37     public async Task<ActionResult<Product>> Get(int id)
38     {
39         var product = await _context.Products.FindAsync(id);
40         return product == null ? NotFound() : Ok(product); // 404 если не найден
41     }
42
43     // =====
44     // POST: /api/Product
45     // Создаёт новый товар
46     // Только для роли Admin
47     // =====
48     [Authorize(Roles = "Admin")] // Требуется авторизация и роли Admin
49     [HttpPost]
50     public async Task<ActionResult<Product>> Create(Product product) // Возвращаем созданный
    ↪ продукт
51     {
52         _context.Products.Add(product); // Добавляем в контекст
53         await _context.SaveChangesAsync(); // Сохраняем изменения
54
55         // Возвращаем статус 201 и маршрут к новому товару
56         return CreatedAtAction(nameof(Get), new { id = product.Id }, product);
57     }
58
59     // =====
60     // PUT: /api/Product/5
61     // Обновляет товар по ID
62     // Только для роли Admin
63     // =====
64     [Authorize(Roles = "Admin")]
65     [HttpPut("{id}")]
66     public async Task<IActionResult> Update(int id, Product product) // IActionResult, т.к.
    ↪ NoContent
67     {
68         if (id != product.Id) return BadRequest(); // Проверка ID
69
70         _context.Entry(product).State = EntityState.Modified; // Помечаем как "изменённый"
71
72         try
73         {
74             await _context.SaveChangesAsync(); // Сохраняем
75         }
76         catch (DbUpdateConcurrencyException)
77         {
78             if (!_context.Products.Any(e => e.Id == id)) // Проверяем, существует ли продукт
79             {
80                 return NotFound();
81             }

```

```
82         else
83         {
84             throw;
85         }
86     }
87     return NoContent(); // 204 – успешно, без тела
88 }
89
90 // =====
91 // DELETE: /api/Product/5
92 // Удаляет товар по ID
93 // Только для роли Admin
94 // =====
95 [Authorize(Roles = "Admin")]
96 [HttpDelete("{id}")]
97 public async Task<IActionResult> Delete(int id) // IActionResult
98 {
99     var product = await _context.Products.FindAsync(id);
100     if (product == null) return NotFound(); // 404 если не найден
101
102     _context.Products.Remove(product); // Удаление
103     await _context.SaveChangesAsync(); // Сохраняем
104
105     return NoContent(); // 204 – успешно удалено
106 }
107 }
108 }
```

Листинг 7.1 – Контроллер `ProductController.cs`

8. ApiService

Класс `ApiService` — это служба клиента WPF-приложения, которая взаимодействует с REST API (Web API на ASP.NET Core). Он отвечает за:

- настройку подключения;
- авторизацию и хранение токена;
- выполнение CRUD-операций для товаров и заказов;
- регистрацию и логин пользователя.

Данный класс нужен, чтобы разгрузить `ViewModel` от сетевых вызовов и предоставить единый интерфейс для общения WPF-клиента с сервером. Это часть архитектуры MVVM.

 **Пояснение с комментариями в коде:**

```
1 // using System.Net.Http;
2 // using System.Net.Http.Headers; // For AuthenticationHeaderValue
3 // using System.Net.Http.Json; // For PostAsJsonAsync, ReadFromJsonAsync
4 // using System.Text.Json; // For JsonSerializer, JsonElement
5 // using System.Threading.Tasks;
6 // using System.Collections.Generic; // For List
7 // Models: Product, Order, UserRegisterDto must be defined
8
9 public class ApiService
10 {
11     private HttpClient _http;           // Клиент для HTTP-запросов
12     private string _token;              // JWT-токен, полученный при логине
13     private string _role;               // Роль пользователя (Admin/Customer)
14 }
```

Инициализация клиента:

```
1 public async Task InitAsync() // Сделаем асинхронным, если TryUseHttps асинхронный
2 {
3     var handler = new HttpClientHandler
4     {
5         // ВАЖНО: В реальном приложении используйте валидные сертификаты!
6         ServerCertificateCustomValidationCallback = (message, cert, chain, errors) => true //
        ↪ Игнорируем SSL в dev
7     };
8
9     // Проверка: работает ли HTTPS?
10    bool useHttps = await TryUseHttpsAsync(); // Назовем его Async
11
12    // Настраиваем базовый адрес
```

```

13     var baseUri = useHttps
14         ? "https://localhost:7259/api/" // Порт HTTPS по умолчанию для ASP.NET Core
15         : "http://localhost:5107/api/"; // Порт HTTP по умолчанию для ASP.NET Core
16
17     _http = new HttpClient(handler)
18     {
19         BaseAddress = new Uri(baseUri)
20     };
21 }
22
23 private async Task<bool> TryUseHttpsAsync() // Пример реализации
24 {
25     try
26     {
27         using var testClient = new HttpClient(new HttpClientHandler {
28             ↪ ServerCertificateCustomValidationCallback = (a,b,c,d) => true });
29         testClient.Timeout = TimeSpan.FromSeconds(3); // Таймаут для проверки
30         var response = await testClient.GetAsync("https://localhost:7259/api/Product"); // Пробный
31             ↪ запрос
32         return response.IsSuccessStatusCode;
33     }
34     catch
35     {
36         return false;
37     }
38 }

```

Метод TryUseHttpsAsync() пытается подключиться по HTTPS и возвращает true, если сервер отвечает.

🔗 Работа с токеном:

```

1 public void SetToken(string token)
2 {
3     _token = token;
4     if (_http != null) // Убедимся, что HttpClient инициализирован
5     {
6         _http.DefaultRequestHeaders.Authorization =
7             new System.Net.Http.Headers.AuthenticationHeaderValue("Bearer", token); // Добавляем
8             ↪ токен
9     }
10 }
11
12 public void SetRole(string role) // Добавлен метод для установки роли
13 {
14     _role = role;
15 }
16
17 public string GetRole() => _role; // Получение роли

```

```
18 public void ClearAuth() // Очистка авторизации
19 {
20     _token = null;
21     _role = null;
22     if (_http != null)
23     {
24         _http.DefaultRequestHeaders.Authorization = null;
25     }
26 }
```

Также можно:

- получить роль `GetRole()`;
- очистить авторизацию `ClearAuth()` (удаляет токен из заголовков).

Работа с продуктами (Product):

```
1 public async Task<List<Product>> GetProductsAsync()
2 {
3     // HttpClient должен быть инициализирован через InitAsync()
4     var response = await _http.GetAsync("Product");
5     response.EnsureSuccessStatusCode(); // Проверка на ошибки HTTP
6     var json = await response.Content.ReadAsStringAsync();
7
8     // Опции десериализации для игнорирования регистра символов в именах свойств
9     var options = new JsonSerializerOptions { PropertyNameCaseInsensitive = true };
10    var products = JsonSerializer.Deserialize<List<Product>>(json, options);
11
12    return products ?? new List<Product>();
13 }
```

Также реализованы:

- `AddProductAsync` — добавляет товар через `POST /api/Product`;
- `UpdateProductAsync` — обновляет товар;
- `DeleteProductAsync` — удаляет товар.

Работа с заказами (Order):

```
1 public async Task<List<Order>> GetOrdersAsync()
2 {
3     if (string.IsNullOrEmpty(_token)) return new List<Order>(); // Без токена нельзя
4
5     var response = await _http.GetAsync("Order");
6     if (response.StatusCode == System.Net.HttpStatusCode.Unauthorized) return new List<Order>();
```

```
7     response.EnsureSuccessStatusCode();
8
9     // Используем ReadFromJsonAsync для упрощения
10    var orders = await response.Content.ReadFromJsonAsync<List<Order>>(
11        new JsonSerializerOptions { PropertyNameCaseInsensitive = true });
12
13    // Фильтрация для удаления null элементов если API может их вернуть
14    return orders?.Where(o => o != null && o.Items != null).ToList() ?? new List<Order>();
15 }
```

Также есть:

- AddOrderAsync(order) — создание заказа;
- DeleteOrderAsync(id) — удаление по ID.

🔗 Аутентификация и регистрация:

```
1 public async Task<bool> LoginAsync(string username, string password)
2 {
3     var dto = new { Username = username, Password = password }; // Анонимный тип для LoginDto
4     var response = await _http.PostAsJsonAsync("auth/login", dto);
5
6     if (!response.IsSuccessStatusCode) return false;
7
8     // Чтение JSON ответа для получения токена и роли
9     var jsonDocument = await JsonDocument.ParseAsync(await response.Content.ReadAsStreamAsync());
10    string token = jsonDocument.RootElement.GetProperty("token").GetString();
11    string role = jsonDocument.RootElement.GetProperty("role").GetString();
12
13    SetToken(token);
14    SetRole(role); // Сохраняем роль
15
16    return true;
17 }
```

Регистрация:

```
1 public async Task RegisterAsync(UserRegisterDto dto) // UserRegisterDto должен быть определен
2 {
3     var options = new JsonSerializerOptions
4     {
5         PropertyNamingPolicy = null // Отключаем camelCase, если API ожидает PascalCase для DTO
6     };
7
8     var response = await _http.PostAsJsonAsync("auth/register", dto, options);
9     response.EnsureSuccessStatusCode(); // Выбросит исключение при ошибке
10 }
```

9. Program.cs

Этот файл — это основная точка входа в ASP.NET Core Web API-приложение. Он находится в файле Program.cs и отвечает за настройку всех компонентов, сервисов, middleware, базы данных и запуск приложения. Ниже приведено его подробное описание и пояснение по блокам. Этот файл конфигурирует:

- регистрацию сервисов (DI);
- подключение базы данных через Entity Framework;
- настройку JWT-аутентификации;
- настройку CORS;
- генерацию Swagger-документации;
- запуск самого веб-сервера.

Полный [листинг](#) данного метода находится в репозитории [GitHub](#) . Далее будет рассмотрена структура данного метода с комментариями.

```
1 // using Microsoft.AspNetCore.Builder;
2 // using Microsoft.Extensions.DependencyInjection;
3 // using Microsoft.EntityFrameworkCore; // Для UseSqlite
4 // using Microsoft.AspNetCore.Authentication.JwtBearer; // Для AddJwtBearer
5 // using Microsoft.IdentityModel.Tokens; // Для TokenValidationParameters
6 // using System.Text; // Для Encoding
7 // using StoreManagerApi.Data; // Предполагаемый namespace для StoreDbContext
8
9 var builder = WebApplication.CreateBuilder(args); // Создаёт и конфигурирует приложение
```

Настройка JWT-аутентификации

```
1 builder.Services.AddAuthentication(options =>
2 {
3     // Указываем, что по умолчанию используется схема JWT Bearer
4     options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
5     options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
6 })
7 .AddJwtBearer(options =>
8 {
9     options.RequireHttpsMetadata = false; // Отключить HTTPS в режиме разработки (в production true)
10    options.SaveToken = true;             // Сохранять токен в HttpContext после валидации
11    options.TokenValidationParameters = new TokenValidationParameters
```

```
12 {
13     ValidateIssuer = false, // Не проверять издателя токена (true для production)
14     ValidateAudience = false, // Не проверять получателя токена (true для production)
15     ValidateLifetime = true, // Проверять срок действия токена
16     ValidateIssuerSigningKey = true, // Проверять подпись токена
17     IssuerSigningKey = new SymmetricSecurityKey( // Ключ должен совпадать с ключом генерации
18         Encoding.UTF8.GetBytes("THIS_IS_MY_SUPER_SECRET_KEY_1234567890")) // Заменить на
19         ↪ безопасный ключ из конфигурации!
20     // ClockSkew = TimeSpan.Zero // Можно установить для точной проверки времени жизни
21 };
22 });
```

Этот блок необходим для правильной обработки токенов, выданных при логине.

Разрешение CORS

```
1 builder.Services.AddCors(options =>
2 {
3     options.AddPolicy("AllowAll", policy =>
4     {
5         policy.AllowAnyOrigin() // Разрешить запросы с любого источника
6         .AllowAnyHeader() // Разрешить любые заголовки
7         .AllowAnyMethod(); // Разрешить любые HTTP-методы (GET, POST, etc.)
8     });
9 });
```

CORS нужен, если клиент (например, WPF, React) обращается к API с другого адреса.

Настройка базы данных

```
1 builder.Services.AddDbContext<StoreDbContext>(options =>
2     options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")
3         ?? "Data Source=store.db")); // Подключение к SQLite, имя строки из
4         ↪ appsettings.json
```

Здесь подключается база данных и настраивается контекст StoreDbContext для Entity Framework Core.

Контроллеры и Swagger

```
1 builder.Services.AddControllers()
2     .AddJsonOptions(options =>
3     {
4         // Игнорировать циклические ссылки (например, Product -> OrderItem -> Product)
```



```
5         options.JsonSerializerOptions.ReferenceHandler =  
            ↪ System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;  
6         options.JsonSerializerOptions.PropertyNamingPolicy = null; // Если хотите PascalCase, а не  
            ↪ camelCase  
7     });  
8  
9     builder.Services.AddEndpointsApiExplorer(); // Для генерации метаданных API  
10    builder.Services.AddSwaggerGen();           // Для UI Swagger
```

⚙ Создание приложения и базы

```
1    var app = builder.Build();  
2  
3    // Создаёт базу данных при первом запуске, если она не существует  
4    using (var scope = app.Services.CreateScope())  
5    {  
6        var services = scope.ServiceProvider;  
7        try  
8        {  
9            var db = services.GetRequiredService<StoreDbContext>();  
10           db.Database.EnsureCreated(); // Создание БД, если ещё не создана  
11        }  
12        catch (Exception ex)  
13        {  
14            var logger = services.GetRequiredService<ILogger<Program>>();  
15            logger.LogError(ex, "An error occurred creating the DB.");  
16        }  
17    }
```

🚀 Middleware и запуск

```
1    if (app.Environment.IsDevelopment())  
2    {  
3        app.UseSwagger(); // Включаем Swagger только в режиме разработки  
4        app.UseSwaggerUI(); // Интерфейс документации Swagger  
5        app.UseDeveloperExceptionPage(); // Страница с деталями ошибок для разработчика  
6    }  
7  
8    app.UseCors("AllowAll"); // Включаем CORS (политика "AllowAll")  
9  
10   // app.UseHttpsRedirection(); // Можно включить HTTPS, если настроен сертификат  
11  
12   app.UseRouting(); // Добавляем маршрутизацию (важно для порядка middleware)  
13  
14   app.UseAuthentication(); // Включаем аутентификацию JWT (проверяет токен)  
15   app.UseAuthorization(); // Проверка авторизации и ролей (проверяет права доступа)  
16  
17   app.MapControllers(); // Маршрутизация контроллеров (api/...)  
18  
19   app.Run(); // Запускает приложение
```

10. Реализация интерфейса

Далее необходимо рассмотреть реализацию интерфейса форм входа и регистрации, а также пользовательского интерфейса программы в зависимости от используемой роли. На рисунках 10.1a – 10.1b приведен интерфейс форм входа (логина) и регистрации. На рисунке 10.1c приведен интерфейс окна с сообщением об успешной регистрации.

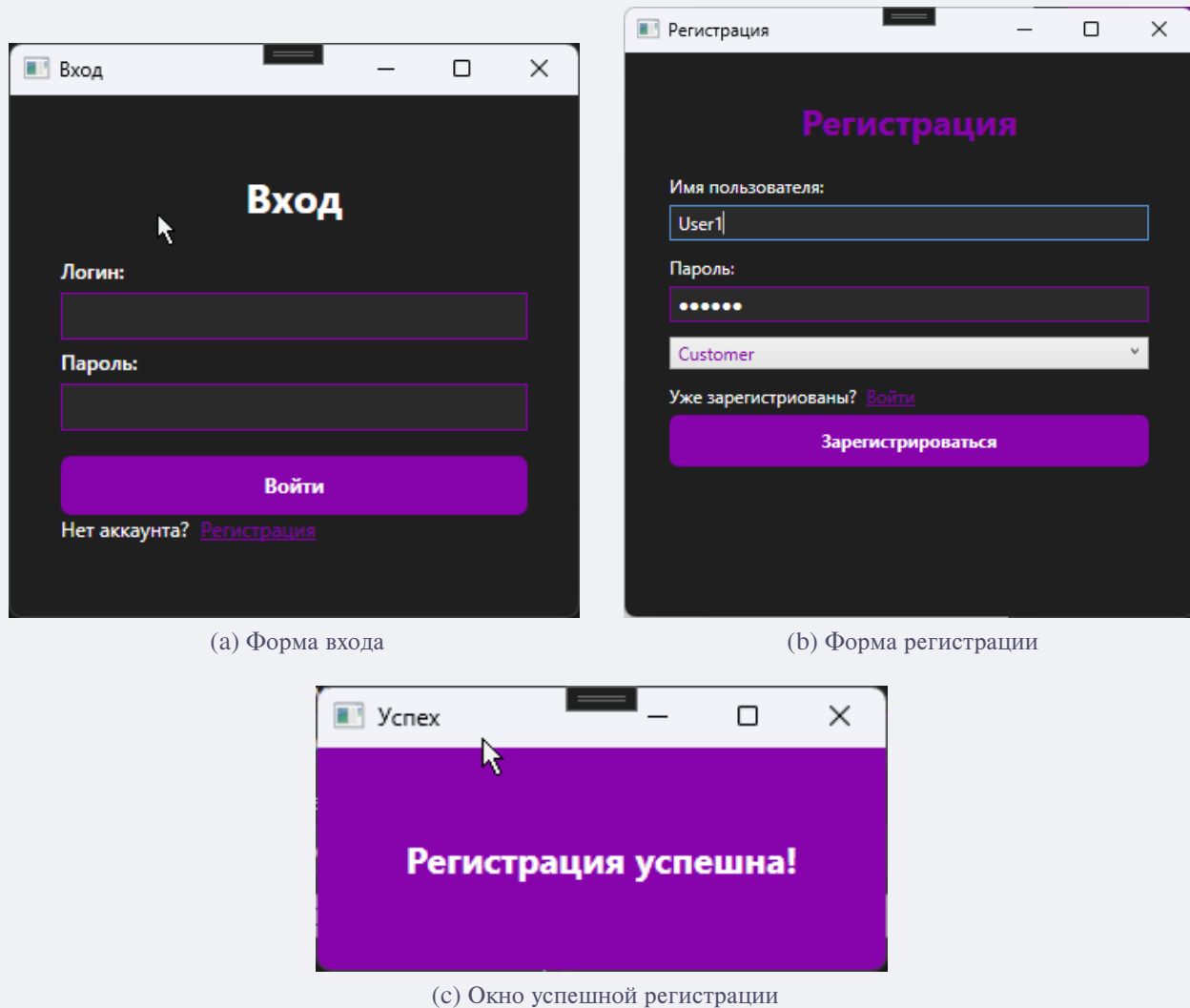


Рисунок 10.1 – Реализация интерфейса входа и регистрации

В форме регистрации реализована возможность выбора роли: пользователь или администратор (Рисунок 10.2).

[NONE]

Рисунок 10.2 – Выбор роли пользователя при регистрации

Интерфейс приложения для роли «Администратор» сохранил возможность добавления, удале-

ния, обновления информации о товарах (Рисунок 10.3).



Рисунок 10.3 – Интерфейс приложения для администратора

Интерфейс приложения для роли «Пользователь» (Рисунок 10.4).

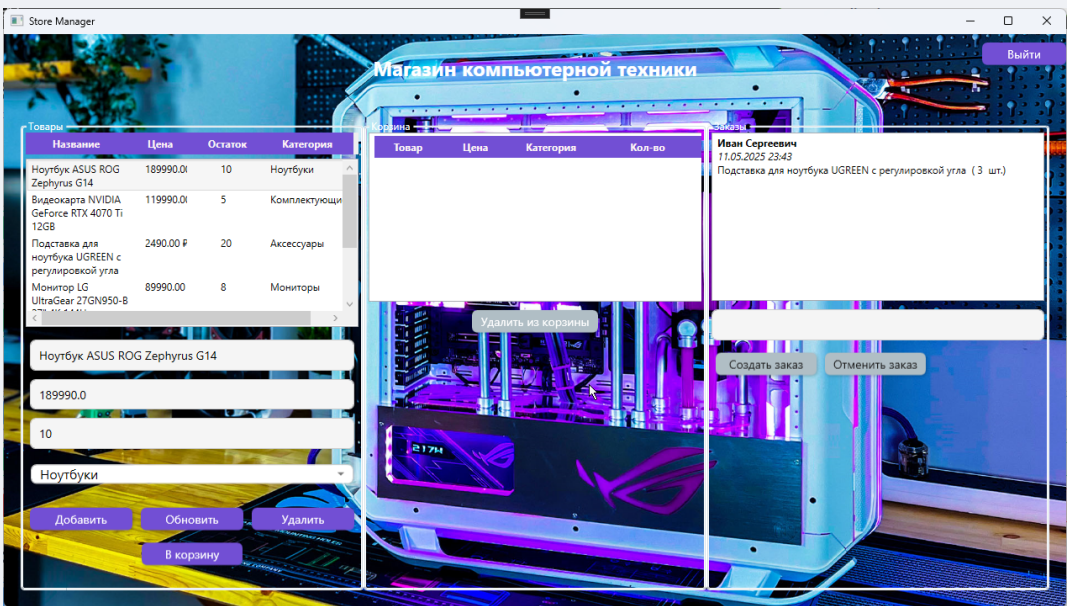


Рисунок 10.4 – Интерфейс приложения для пользователя

11. Вопросы по лабораторной работе №7

Аутентификация и регистрация

1. Как реализуется регистрация нового пользователя в системе?
2. Почему важно хешировать пароль перед сохранением в базу данных?
3. Как работает проверка пароля при входе?
4. Что такое DTO и зачем они используются при регистрации и входе?

JWT и авторизация по ролям

5. Как формируется JWT-токен при авторизации пользователя?
6. Какие claims входят в JWT и для чего они используются?
7. Как система определяет, имеет ли пользователь доступ к определённым действиям?

Архитектура и защита данных

8. Почему нельзя использовать модель User напрямую в методе регистрации?
9. В чём преимущества разделения моделей и DTO?

Клиентская часть (WPF)

10. Как реализована форма входа и регистрации в WPF?
11. Как определяется текущая роль пользователя в клиентском приложении?
12. Чем отличается интерфейс пользователя и администратора?

MVVM и взаимодействие с API

13. Как ViewModel взаимодействует с ApiService?
14. Почему используется команда RegisterCommand в RegisterViewModel?
15. Какие действия выполняет метод RegisterAsync()?